

uocsclub.net/talks/vim26

Installation
============

Windows:
  www.vim.org/download.php

Mac:
  type "vim" in your terminal

Anything else:
  I'm sure you can figure it out

reminder to me: turn on key display

Why use Vim?
============

- "edit at the speed of thought"

- no mouse!

- do you use Control-C?

- on most unixen, "standard" keybinds, and is lightweight

How to learn Vim?
=================

- jump right in!

- your favourite editor probably has emulation

- vimtutor

Vim's history

QED
  - Butler Lampson, L Peter Deutsch, 1966, SDS 940
  - line editor designed for teleprinters

ed(1)
  - Ken Thompson, 1969, PDP-7
  - part of the POSIX standard

ex
  - Bill Joy, 1976

vi
  - Bill Joy, 1979, Unix
  - originally the visual mode of ex

vim
  - Bram Moolenaar, 1988, AmigaOS

Let's get started!

Download uocsclub.net/talks/vim26/example.html

Fire up Vim!!

```
vim vim-wikipedia.txt
```

If ever there is a problem or something you do not understand, PLEASE RAISE YOUR HAND!!

Me or someone else will be happy to help! Following along is the best way to learn, and I talk too much.

This will be an infodump on purpose.

Modes
=====

Vim is a modal editor. The mode is displayed in the bottom left.
         ----

 k
h l moves around
 j

i enters insert mode, where you can edit stuff.
When in doubt, hit "escape".

:w<CR> to save

x deletes the character under the cursor.

u undo, CTRL-R redo

Take some time to practise these! You'll use them often.

Motions (:h motion.txt)
=======

hjkl are motions!

```
w       W        forwards to a word/WORD
b       B        backwards to a word/WORD
e       E        to the end of the next word/WORD
gg      G        to the top/bottom of file
0       $        to the start/end of a line
^                to the first non-whitespace character of the line
f{char} F{char}  forward/backward until you FIND {char}
t{char} T{char}  forward/backward TILL {char}
;       ,        repeat last f, t, F, or T in the same/opposite direction
/       ?        search forwards/backwards
  this can be suffixed with /e to go to the end, like /pattern/e
n       N        repeat last search in the same/opposite direction
}       {        to next/previous paragraph
```

All motions can be prefixed by a number. eg. 2w 10k

```
Operators (:h operator)
=========

d   delete
c   change                    +--------------------------------------+
gU  UPPERCASE                 | mini-tip: ~ to toggle case of a char |
gu  lowercase                 +--------------------------------------+
g~  toggle case

The (hopefully not) secret sauce:
operator + motion

Double to operator on a line (dd, cc)

D and C operate from cursor till end of line.
```

Text Objects (:h text-objects)
============

Combine operator + text object!

a = "a", i = "inner"

```
w W        word/WORD                      +--------------------+
p          paragraph                      | mini-tip: . repeats |
()b {}B    () or {}  block                +--------------------+
[] <> "'`  other delimiters
t          tag block

eg.                                  +---------------------------------+
 ci" = "delete inner string"         | bonus text object:              |
 dap = "delete a paragraph"          | gn   a search result            |
 cib = "change inside parentheses"   +---------------------------------+
 dit = "delete inner tag"
```

These will even jump to the next instance!

Insert mode revisited
=====================

i insert before cursor
a append after cursor

These can also be capitalized just like d and c!

So:

I insert at beginning of line
A append at end of line

+-----------------------------+
| mini-tip: r replaces a char |
+-----------------------------+

o O add line before/after then enter insert mode

Copy paste! and visual modes

Introducing a new operator: y ("yank", what vim calls copy)
 eg. yaw, yy, yib

p P paste after/before

The delete operator also copies to clipboard

System clipboard: "+p to paste from, "+y to copy to

New mode unlocked:

v	visual mode
V	visual line mode
CTRL-V	visual block mode

While in visual mode, can use text objects, and operators (take that Kakoune!)

More file navigation
====================

CTRL-I CTRL-O move forward/backward in jumplist (:h jumplist)
CTRL-D CTRL-U scroll down/up by half page
CTRL-F CTRL-B scroll down/up by full page

zt zz zb   put cursor at top, centre, bottom

gi gv   go to last place insertion stopped, select last selection

* # search forward/backward for word nearest to the cursor

% jump to matching bracket

```
Editing multiple files (:h windows.txt)
======================

:e <filename>    edit file
:b <buffername>  switch to buffer
:bn :bp          next, previous buffer

:vs CTRL-W_v  vertical split
:sp CTRL-W_s  horizontal split

CTRL-W_h CTRL-W_j CTRL-W_k CTRL-W_l  switch focused window

:q   close window
:bd  close buffer and window
```

```
Reading help pages (:h help-summary)
==================

very important - I wish I had learned these commands sooner

:h open a help page
 :h *
 :h :h
 :h CTRL-R
 :h CTRL-A       <- CTRL-A in normal mode
 :h i_CTRL-A     <- CTRL-A in insert mode

You can tab complete these, or use CTRL-D to see all completions.

CTRL-]  follow link
CTRL-O  go back (we've seen this before!)

:helpgrep :cope  search help files

quickfix is really cool! (:h quickfix)  :cn  :vimgrep
```

```
Find-and-replace (:h :s)
================

:s/pattern/replacement/flags

use :%s for global (% comes from ed and means "whole file")

parts can be dropped off the end as desired.
a lone :s repeats, and :s//replacement/flags repeats the last pattern

useful flags:
 g  global, replace all occurances on line
 c  confirm, prompt before changing
 n  replace NOTHING, good for counting

magic:
 ~ in pattern part means last substitute string
 & in replacement part means the matched text
```

```
Registers (:h registers)
=========

Remember "+p? This tells vim to use the '+' register. Think of these as
variables. (for help, prefix with quote. eg. :h quote=)

Some of these Vim reserves, we can use a-zA-Z as we wish.

"ap

""              unnamed register
"=              expression register
"0              most recent yank register
"-              small delete register
"1 "2 ... "9    deleted/changed registers for at least one line

for reading help pages, register names are prefixed with quote
 :h quote=

i_CTRL_R  insert register
```

```
Macros (:h q)
======

import dog0 from './dog0.jpg';

q{char}  start recording to register {char}
q        end recording
@{char}  execute register {char}
@@       execute last executed register

+-----------------------------------------------------------------------+
| mini-tip: i_CTRL-V inserts a special key such as escape or enter |
+-----------------------------------------------------------------------+
```

```
Marks (:h mark-motions)
=====

Yet another use for registers! Marks store places.

Lowercase (a-z) are buffer-specific, uppercase (A-Z) are global.

m{char}          create mark
'{char} `{char}  motion to go to line/char where the mark is

' and ` count as jumps for CTRL-O/CTRL-I

Useful marks:
'^     place where insert mode was exited (gi<ESC> uses this)
'< '>  beginning/end of last visual mode selection
```

```
Configuring Vim (:h vimrc)
==========

:e ~/.vimrc

if on Windows:  :e ~/_vimrc

Put all config here! Everything you put here is basically run as if you
typed it prefixed with a colon.

eg.

syntax on   " enable syntax highlighting
set number  " enable line numbers

All options (things that are set) can be found in  :h options.txt
```

Quitting Vim
============

:q   quit

:wq   save and quit

:qa   quit all

:q!   quit, discarding unsaved changes

:wqa save all and quit (a good catch-all)

Learning more

- Read the Vim help pages!

- www.youtube.com/@emilien.breton

- vim.fandom.com/wiki/Vim_Tips_Wiki

Happy Vimming!